

Regex for coders what is it and how does it works

Regex, short for "regular expression," is a way of writing patterns that describe text. Instead of searching for an exact string, you write a pattern that matches a whole category of strings that share some structure, which makes it useful for validation, searching, and text manipulation.

The basic idea is that you build a small expression out of literal characters and special symbols, and that expression gets compared against a target string to see where it matches. For example, the pattern `cat` just matches the literal text "cat" wherever it appears. But the real power comes from the special symbols that let you describe patterns rather than exact text.

A few of the most common building blocks: `.` matches any single character, `*` means "zero or more of the previous thing," `+` means "one or more," and `?` means "zero or one" (making something optional). Square brackets like `[abc]` match any one character inside them, and `[a-z]` matches any lowercase letter using a range. The caret `^` and dollar sign `$` anchor a match to the start or end of a line. Parentheses `(...)` group parts of a pattern together, often so you can capture that piece of text separately or apply a repetition operator to the whole group. The pipe `|` means "or," so `cat|dog` matches either word.

There are also shorthand character classes: `\d` for any digit, `\w` for any "word" character (letters, digits, underscore), and `\s` for whitespace. So a pattern like `\d{3}-\d{4}` would match something like "555-1234," since `{3}` means "exactly three of the previous thing."

Mechanically, when a regex engine runs, it tries to match the pattern against the text starting at each position, moving left to right, backtracking when needed if a tentative match fails partway through. Most languages give you a regex engine through a built-in module: Python has `re`, JavaScript has regex literals like `/pattern/flags` along with `RegExp`, Java has `java.util.regex`, and so on. The syntax is fairly consistent across languages with small variations in features and escaping rules.

A simple example in Python:

```
python

import re

text = "My phone number is 555-1234"
match = re.search(r"\d{3}-\d{4}", text)
if match:
    print(match.group()) # "555-1234"
```

Common use cases include validating input formats like emails or phone numbers, searching and replacing text (like swapping all instances of a date format), parsing log files, and extracting specific pieces of data from larger blocks of text.

One thing worth knowing early on: regex can get hard to read fast, especially with nested groups and lots of special characters, so it's good practice to comment complex patterns or break them into named pieces when your language supports that. If you want, I can walk through a specific pattern you're trying to write, or build out some practice examples in whatever language you're using.

Teach me regex

I'll teach you regex from the ground up, building piece by piece so each new idea has something to attach to. Let's start with the absolute basics and work toward more advanced patterns.

1. Literal matching

The simplest regex is just plain text. The pattern `hello` matches the literal characters "hello" wherever they appear in a string. No magic yet, just exact text.

2. The dot `.`

`.` matches any single character except a newline (in most engines, by default). So `c.t` matches "cat," "cot," "c9t," anything with one character between c and t.

3. Character classes `[ ]`

Square brackets let you match any one character from a set. `[abc]` matches "a," "b," or "c." You can use ranges: `[a-z]` matches any lowercase letter, `[0-9]` matches any digit. Putting a `^` right after the opening bracket negates it: `[^0-9]` matches any character that is NOT a digit.

4. Shorthand classes

Some character classes are so common they get their own symbols:

- `\d` = digit, same as `[0-9]`
- `\w` = word character, same as `[A-Za-z0-9_]`
- `\s` = whitespace (spaces, tabs, newlines)
- Capitalizing negates them: `\D`, `\W`, `\S` mean "not digit," "not word char," "not whitespace"

5. Quantifiers

These control how many times something repeats:

- `*` = zero or more
- `+` = one or more
- `?` = zero or one (optional)
- `{n}` = exactly n times
- `{n,m}` = between n and m times
- `{n,}` = n or more times

So `\d+` matches one or more digits in a row, like "42" or "1999." And `colou?r` matches both "color" and "colour" because the `u` is optional.

6. Anchors

`^` means "start of string/line," `$` means "end of string/line." So `^Hello` only matches if "Hello" is at the very beginning, and `end$` only matches if "end" is at the very end.

7. Groups and alternation

Parentheses `( )` group parts of a pattern together. The pipe `|` means "or." So `(cat|dog)` matches either "cat" or "dog." Groups also let you capture pieces of a match to extract later, which is huge for parsing.

Let's check that this is sinking in before I go further into things like non-capturing groups, lookaheads, and greedy vs lazy matching.

Q: Want to try a few practice patterns based on what we just covered before moving on?

A: Yes, give me practice problems

Q: Which language do you want examples in?

A: JavaScript

Good, let's lock in what you've learned with some practice. I'll give you a string and a goal, you write the regex, and I'll check it. We'll use JavaScript syntax (`/pattern/flags`).

Problem 1

String: `"My favorite numbers are 7, 42, and 256"`

Goal: write a regex that matches all the numbers in this string.

Try writing the pattern, something like `/your pattern here/g` (the `g` flag means "global," find all matches, not just the first one).

Problem 2

String: `"cat, bat, hat, mat, fat"`

Goal: match any of these words that rhyme with "at" but start with a single letter, e.g. match "cat," "bat," "hat," etc.

Problem 3

String: `"color, colour, flavor, flavour"`

Goal: write one pattern that matches both the American and British spelling of these words (using the optional `?` quantifier).

Give me your attempts at all three (or however many you want to try), even if you're not fully sure, and I'll walk through what's right, what's off, and why.

`\w` matches special chars??

Good question, and yes, kind of, but let's be precise about what counts as "special."

`\w` matches what's called a "word character," which is specifically: letters (a-z, A-Z), digits (0-9), and the underscore `_`. That's it. It does NOT match spaces, punctuation like `.` `,` `!` `?`, or symbols like `@` `#` `$`.

So if you're looking at something like an email address

`john_doe@example.com`, `\w` would match the letters, digits, and the underscore in `john_doe`, but it would NOT match the `@` or the `.`. Those are not word characters.

The underscore is the one that trips people up, since it feels like punctuation but regex treats it as part of "word" characters, probably because underscores are commonly used inside variable names and identifiers in code (like `my_variable_name`), so it made sense to lump it in with letters and digits for matching purposes.

Quick way to remember it: `\w` is shorthand for the character class `[A-Za-z0-9_]`, exactly equivalent. Anything not in that bracket set is not matched by `\w`, and would instead be matched by its negation, `\W`.

Did you run into this while working on one of the practice problems? If you show me what you tried, I can point out exactly where `\w` did or didn't do what you expected.

`\w+`

`\w+` means "one or more word characters in a row." Since `\w` matches letters, digits, and underscores, `\w+` will grab a whole consecutive run of those, stopping the moment it hits anything else, like a space, punctuation, or symbol.

So if you ran `\w+` against `"Hello, world! my_var = 42"`, with the global flag (`/\w+/g`), it would find: `Hello`, `world`, `my_var`, `42`. Notice it grabs `my_var` as one single match because the underscore counts as a word character, but it splits at the comma, the space, the exclamation mark, and the equals sign, since none of those are word characters.

This makes `\w+` one of the most commonly used patterns in regex, since it's basically a quick way to say "match a word" or "match an identifier," in casual everyday text or in code.

One subtlety worth flagging: `\w+` is greedy by default, meaning it grabs as many word characters as it possibly can before giving up if needed. We haven't covered greedy vs lazy matching yet, but it's relevant here, want me to fold that into your next practice round, or finish checking the three problems I gave you first?

only `\w`

`\w` by itself (no quantifier) matches exactly one word character, nothing more.

IPv4 matching regex

So if you ran `/\w/` against `"Hi there!"`, it would match just the single letter `H`, the very first word character it finds, and stop. It wouldn't grab "Hi" as a whole, just that one `H`.

If you add the global flag, `/\w/g`, against `"Hi there!"`, it would find every individual word character one at a time: `H`, `i`, `t`, `h`, `e`, `r`, `e`. Seven separate single-character matches. It skips the space and the `!` since those aren't word characters, but it does NOT group consecutive ones together into "Hi" and "there," because there's no `+` or `*` telling it to keep going.

That's really the core difference: `\w` = exactly one word character, `\w+` = one or more in a row (grouped together as a single match), `\w*` = zero or more (so it could even match an empty string at a position with no word characters).

This is a good moment to test it. Want to try predicting what `/\w/g` versus `/\w+/g` would each return against the string `"a1 b22 c333"`, just to make sure the difference is fully clicked into place?

Let's build this one step by step rather than just handing you a finished pattern, since it's a great exercise in applying everything we've covered so far, plus a couple of new ideas.

The naive first attempt

An IPv4 address looks like four numbers (0-255) separated by dots, e.g. `192.168.1.1`. Your first instinct, using only what we've covered, might be something like:


```
\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}
```

Let's break that down: `\d{1,3}` means "1 to 3 digits," and `\.` means a literal dot (we need the backslash because a bare `.` means "any character," so we escape it to mean an actual period). Repeat that pattern four times separated by `\.`.

Try it yourself first

Before I tell you whether that's "correct," try testing it mentally (or in a JS console) against these three strings:

1. `192.168.1.1` (should match)
2. `999.999.999.999` (should this match? think about what `\d{1,3}` allows)
3. `12.34.56` (should NOT match, only 3 numbers)